	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering (PCCoE)** (An Autonomous Institute) Affiliated to Savitribai Phule Pune University(SPPU) ISO 21001:2018 Certified by TUV	

Course: Operating Systems Laboratory	**Code:** BIT26PC02
Name: Amar Vaijinath Chavan	**PRN:** 124B2F001
Assignment 3: Process Creation, Demonstration of Zombie and Orphan States	

Aim

To implement process creation in Linux using the fork() system call, where the parent and child processes sort given integers using sorting algorithms, and to demonstrate Zombie and Orphan process states using the wait() system call.

Objectives

- To understand the fork(), wait(), and exit() system calls in a Linux environment.
- To study the hierarchy of parent and child processes.
- To analyze the behavior of processes when synchronization (via wait()) is absent or handled differently.
- To observe the transition of process states in the system process table.

Theory

1. Process Creation (fork)

The fork() system call creates a new process (child) by duplicating the calling process (parent).

- It returns **0** to the child process.
- It returns the **PID of the child** to the parent process.
- It returns a **negative value** if the creation fails.

2. Orphan Process

An Orphan process occurs when a parent process terminates before its child process. When this happens, the child process is "adopted" by a system initialization process (usually init with PID 1 or a user-session reaper), ensuring that every process has a parent.

3. Zombie Process

A Zombie process is a process that has completed execution (via exit) but still has an entry in the process table. This happens because the parent has not yet read the child's exit status using the wait() system call. The process stays in a "defunct" state, consuming only a small amount of system memory for the process table entry.

Code 1: Non Orphan Process Demonstration

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    int n, i, j, temp;
    int arr[20];
    pid_t p;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Before fork: pid=%d ppid=%d\n", getpid(), getppid());

    p = fork();

    if (p == 0) {
        // CHILD PROCESS
        printf("\nChild Process: Ascending Order\n");

        for (i = 0; i < n - 1; i++) {
            for (j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        for (i = 0; i < n; i++)
            printf("%d ", arr[i]);

        printf("\nChild pid=%d ppid=%d\n", getpid(), getppid());
    }
    else {
        // PARENT PROCESS
        wait(NULL);  // Parent waits for child

        printf("\nParent Process: Descending Order\n");


 for (i = 0; i < n - 1; i++) {
            for (j = 0; j < n - i - 1; j++) {
                if (arr[j] < arr[j + 1]) {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
```

```
                arr[j + 1] = temp;
            }
        }
    }

    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nParent pid=%d ppid=%d\n", getpid(), getppid());
  }

  return 0;
}
```

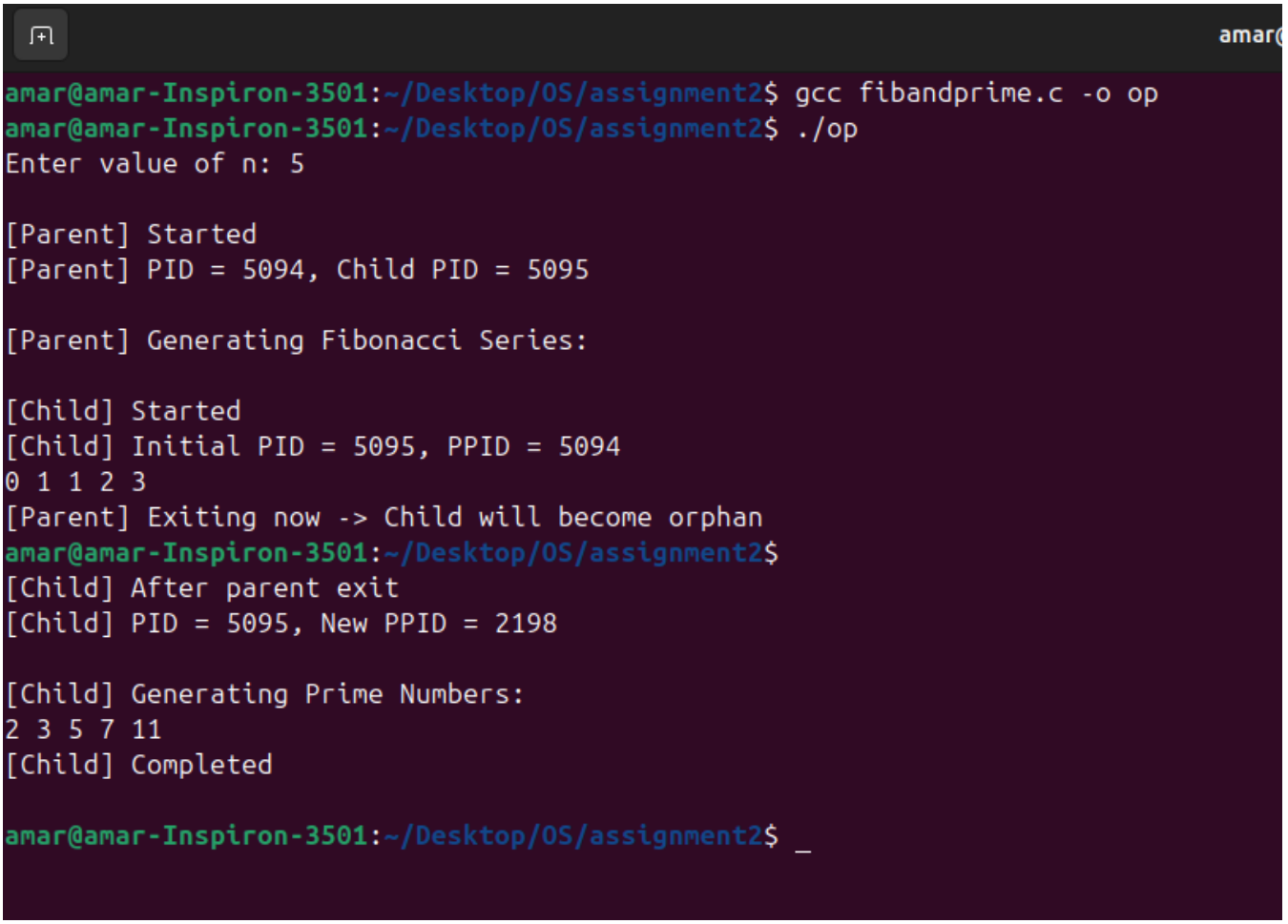
```
amar@amar-Inspiron-3501:~/Desktop/OS/assignment2$ gcc sort.c -o sort
amar@amar-Inspiron-3501:~/Desktop/OS/assignment2$ ./sort
Enter number of elements: 5
Enter elements:
1
5
3
2
4
Before fork: pid=5411 ppid=5322

Child Process: Ascending Order
1 2 3 4 5
Child pid=5412 ppid=5411

Parent Process: Descending Order
5 4 3 2 1
Parent pid=5411 ppid=5322
amar@amar-Inspiron-3501:~/Desktop/OS/assignment2$
```

Code 2: Orphan Process Demonstration

Description:

The parent process generates Fibonacci numbers and terminates using exit(), while the child process sleeps for 5 seconds and then generates prime numbers. Since the parent terminates before the child finishes, the child becomes an **Orphan process** and its parent ID changes to the system process.

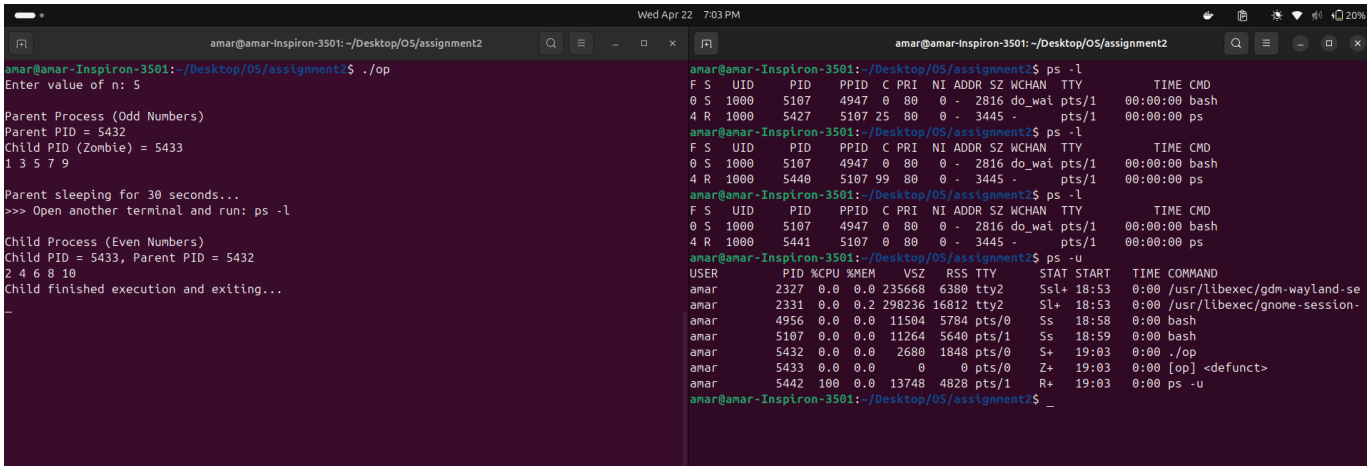
```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main(void) {
   int n, i, j, count = 0;
   pid_t p;

   printf("Enter value of n: ");
   scanf("%d", &n);

   p = fork();

   if (p == 0) {
      // CHILD PROCESS
      printf("\n[Child] Started\n");
      printf("[Child] Initial PID = %d, PPID = %d\n", getpid(), getppid());

      sleep(6);  // Wait for parent to exit

      // After parent exits, PPID should change
      printf("\n[Child] After parent exit\n");
      printf("[Child] PID = %d, New PPID = %d\n", getpid(), getppid());

      printf("\n[Child] Generating Prime Numbers:\n");

      int num = 2;
      while (count < n) {
         int prime = 1;
         for (j = 2; j <= num / 2; j++) {
            if (num % j == 0) {
               prime = 0;
               break;
            }
         }
         if (prime == 1) {
            printf("%d ", num);
            count++;
         }
         num++;
      }
```


amar@amar-Inspiron-3501:~/Desktop/OS/assignment2$ gcc fibandprime.c -o op
amar@amar-Inspiron-3501:~/Desktop/OS/assignment2$ ./op
Enter value of n: 5
[Parent] Started
[Parent] PID = 5094, Child PID = 5095
[Parent] Generating Fibonacci Series:
[Child] Started
[Child] Initial PID = 5095, PPID = 5094
0 1 1 2 3
[Parent] Exiting now -> Child will become orphan
amar@amar-Inspiron-3501:~/Desktop/OS/assignment2$
[Child] After parent exit
[Child] PID = 5095, New PPID = 2198
[Child] Generating Prime Numbers:
2 3 5 7 11
[Child] Completed
amar@amar-Inspiron-3501:~/Desktop/OS/assignment2$ _

Code 3: Zombie Process Demonstration

Description: The child generates even numbers and terminates, while the parent generates odd numbers. The parent does not call wait() and sleeps for 30 seconds. The child remains in the process table as a **Zombie process** until the parent terminates.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main() {
    int n, i;
    pid_t p;

    printf("Enter value of n: ");
    scanf("%d", &n);

    p = fork();

    if (p == 0) {
        // Child process
        printf("\nChild Process (Even Numbers)\n");
        printf("Child PID = %d, Parent PID = %d\n", getpid(), getppid());

        for (i = 1; i <= n; i++)
            printf("%d ", 2 * i);

        printf("\nChild finished execution and exiting...\n");
        exit(0);
    }
    else {
        // Parent process
        printf("\nParent Process (Odd Numbers)\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID (Zombie) = %d\n", p);

        for (i = 1; i <= n; i++)
            printf("%d ", 2 * i - 1);

        printf("\n\nParent sleeping for 30 seconds...\n");
        printf(">>> Open another terminal and run: ps -l\n");

        sleep(30);

        printf("\nParent exiting\n");
    }

    return 0;
}
```



Conclusion

From this assignment, we conclude:

1. **Process Hierarchy:** fork() successfully creates a concurrent execution path.
2. **Synchronization:** The wait() system call is essential for the parent to synchronize with the child and collect its exit status.
3. **State Management:** An **Orphan** process is safely handled by the system (re-parented), whereas a **Zombie** process exists because the parent fails to acknowledge the child's termination, highlighting the importance of proper process management in system programming.